

Artificial Intelligence Exam Notes

CT 710 (BEI, IV-I) · CT 653 (BCT, III-II) · CT 78506 (BEX, IV-II) · TU, IOE · tiers over 41 papers, 2069–2082 BS

7. Applications of AI

14 hrs · asked in **all 41 papers** · 3 syllabus groups, 12 leaf topics · **893 marks, 26.3% of the archive** · **176 numerical (20%) + 717 theory** · the gate networks, the Hopfield matrix, the forward propagation and the parse tree are worked in *Ch7 – Applications of AI – Solved Problems*.

Read this first:

- **This one chapter is a quarter of the paper.** It is bigger than chapters 1, 2, 3 and 5 put together, and **no paper has ever skipped it**. Expect three or four questions from it every time.
- **Four questions dominate, and each is nearly certain:**
 - The steps of NLP, with a block diagram **TOP 23/41** – the most-asked question in the entire subject.
 - Perceptron, and design a net for AND / OR / XOR **TOP 17/41**.
 - Expert system architecture, with a block diagram **TOP 17/41**.
 - Machine vision: role, steps, applications **TOP 12/41**.
- **Three drawings are worth more than anything else here:** the five-box NLP pipeline, the four-box expert system, and a two-input perceptron. Practise all three until you can draw them in one minute each.
- **Then the neural networks:** Kohonen **TOP 9/41**, Hopfield **TOP 8/41**, back-propagation **TOP 8/41**, the neuron model and activation functions **TOP 8/41**, Hebbian learning **HOT 4/41**.
- **The examiner pairs across sub-topics**, so revise them together: *machine vision + back-propagation* (71 Ma, 72 Ash), *NLP challenges + machine vision* (79 Bh), *declarative Vs procedural + expert system architecture* (70 Ma, 72 Ma, 73 Ma).
- **Chapter 6's neural-network syllabus line is answered here.** The syllabus files neural networks under Machine Learning, but every paper sets them under Applications of AI, so perceptron, back-propagation, Hopfield, Kohonen and Hebbian learning are all in this chapter.

7.1 The artificial neuron: model, activation function, ANN Vs the brain

What is an ANN? Explain the McCulloch-Pitts model with a diagram. What is the role of the activation function? 4 3+4 4+3 1+7 3+5 8 10

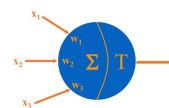
TOP 8/41 71 Bh 3+5, 81 Ch 3+4, 82 Ka 4+3, 75 Ba 8, 76 Ba 10, 80 Bh 2+2+4, 79 Bh 1+7, 69 Bh 4 · 71 Bh and 69 Bh want the brain comparison, 81 Ch and 82 Ka the activation function, 75 Ba and 76 Ba the MP neuron on AND and XOR (worked in the companion), 80 Bh the mathematical model and what training means

Definition and the mathematical model

- An artificial neural network is a parallel computational system of many simple processing elements (neurons), connected in a specific way, that learns a task from data rather than from a program.
- Modelled on the brain: massively parallel, and **tolerant of noise and of the failure of individual units**.
- One neuron does three things, and that is the whole model:
 - 1. **Weight** each input. w_1x_1 up to w_nx_n . The weight is the strength of that connection, and **the weights are what the network learns**.
 - 2. **Sum** them, plus a **bias** b (equivalently, subtract a **threshold** θ):

$$net = \sum_{i=1}^n w_i x_i + b$$

- 3. **Apply an activation function** f to that sum: $y = f(net)$.



The neuron fires if the following inequality is true:

$$x_1w_1 + x_2w_2 + x_3w_3 > T$$

The McCulloch-Pitts neuron and its firing condition. Weight, sum, threshold: the same three operations every neuron in this chapter performs.

- The five elements to name if the question says “elements of a neural network”: **inputs, weights, threshold or bias, activation function, learning rate** α .
- What “**training a network**” means (80 Bh 2): presenting examples and adjusting the weights and biases so the output moves towards the target. **Nothing else in the network changes.**
- **Learning rate** α sets the step size. Too large and the weights oscillate and never settle; too small and training takes forever. Typical 0.01 to 0.5.

The McCulloch-Pitts neuron (1943)

the first neuron model, and the one four papers name.

- Output is **binary**: 1 (fires) or 0. Nothing in between.
- Every input is weighted; the neuron **fires when the weighted sum exceeds the threshold** T :

$$x_1w_1 + x_2w_2 + x_3w_3 > T \Rightarrow y = 1$$

- **Excitatory** inputs carry positive weights, **inhibitory** inputs negative ones, and in the original model **one active inhibitory input vetoes the neuron outright.**
- **The weights are fixed by the designer, not learnt.** That is the difference from a perceptron, and it is the sentence the examiner is looking for.
- It can realise **AND, OR and NOT** – so any Boolean function, if you are allowed several layers – but **one MP neuron cannot do XOR.** Both worked in the companion.

The activation function, and how to choose one (81 Ch 3, 82 Ka 3)

- What it is for, three answers, all worth marks:
 - **Decides whether the neuron fires**, by mapping an unbounded sum onto a bounded output.
 - **Introduces non-linearity.** Without it, a stack of layers collapses into one linear map, and the network can only ever learn a straight line – this is the answer to 79 Bh’s “how can non-linearity be modelled through an ANN”.
 - Must be **differentiable** for gradient descent, which is why back-propagation needs sigmoid or tanh, not a step.
- **Choosing one for a project** – answer with the rule, then an example:
 - **Binary classification output** → **sigmoid**, because it gives a value in $(0, 1)$ readable as a probability.
 - **Multi-class output** → **softmax**.
 - **Regression output** → **linear**, so the value is unbounded.
 - **Hidden layers of a deep net** → **ReLU**: cheap, and it does not saturate, so it does not kill the gradient. Its cost is **dead units** – a neuron whose input stays negative has zero gradient and never recovers, which is exactly what happens in the numerical worked in the companion.
 - **Zero-centred hidden layer** → **tanh**, which converges faster than sigmoid for that reason.
 - **Hard threshold** (step, signum) → only for a perceptron or an MP neuron, never where gradients are needed.

Activation function	Equation	Example	1D Graph
Unit step (Heaviside)	$\phi(z) = \begin{cases} 0, & z < 0, \\ 0.5, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Sign (Signum)	$\phi(z) = \begin{cases} -1, & z < 0, \\ 0, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Linear	$\phi(z) = z$	Adaline, linear regression	
Piece-wise linear	$\phi(z) = \begin{cases} 1, & z \geq \frac{1}{2}, \\ z + \frac{1}{2}, & -\frac{1}{2} < z < \frac{1}{2}, \\ 0, & z \leq -\frac{1}{2}, \end{cases}$	Support vector machine	
Logistic (sigmoid)	$\phi(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, Multi-layer NN	
Hyperbolic tangent	$\phi(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	Multi-layer Neural Networks	
Rectifier, ReLU (Rectified Linear Unit)	$\phi(z) = \max(0, z)$	Multi-layer Neural Networks	
Rectifier, softplus	$\phi(z) = \ln(1 + e^z)$	Multi-layer Neural Networks	

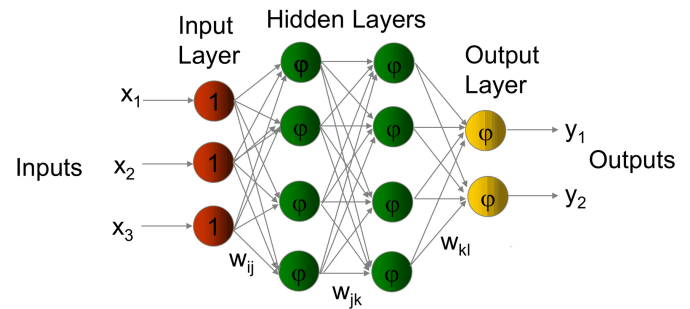
Copyright © Sebastian Raschka 2016
(<http://sebastianraschka.com>)

ANN Vs the human brain (71 Bh 5, 69 Bh 4)

	Human brain	ANN
Units	about 86 billion neurons	a few tens to a few billion
Connections	about 100 trillion synapses	set by the architecture
Speed	slow: a neuron fires at a few hundred Hz	fast: gigahertz
Processing	massively parallel , all at once	parallel in principle, usually serial on one machine
Memory	about 2.5 petabytes, content addressable	limited, address based
Fault tolerance	very high : neurons die daily and nothing is lost	high within a net, none if the machine fails
Learning	continuous, from few examples	needs many examples and retraining
Power	about 20 W	kilowatts to train

– **The contrast worth quoting**: for vision the brain is worth a thousand supercomputers; for arithmetic a pocket calculator is worth ten brains. Structure, not speed, is what differs.

Network structures



- **Feed-forward** – signals travel one way, input to output, with **no loops**. A layer's output never affects that same layer.
 - **Single layer**: input straight to output. Only linearly separable problems (§7.3).
 - **Multilayer (MLP)**: one or more **hidden layers** between them. **One hidden layer is enough to represent any Boolean function**, and with enough units, any continuous function.
- **Feedback (recurrent)** – loops allowed, so the network has **state, and therefore memory**. More powerful, harder to train. Hopfield (§7.5) is the standard example.
- **Competitive** – output units compete and only the winner fires. Kohonen (§7.6).

7.2 Perceptron, linear separability and the XOR problem

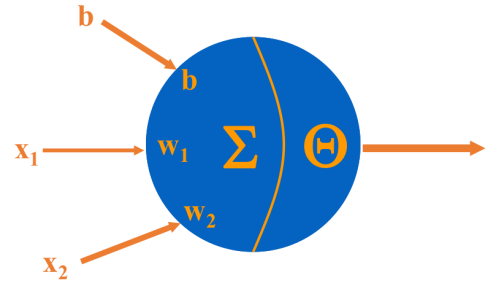
What is a perceptron? Design a neural network that acts as an AND / OR / XOR gate. 4 6 7 10 1+7
2+6 1+8

TOP 17/41 72 Ma 2+7, 73 Ma 1+7, 74 Ma 10, 74 Bh 1+8, 82 Bh 2+6, 79 Ch 2+6, 79 Ash 1+8, 77 Ch 2+6, 79 Jth 1+6, 73 Bh 2+6, 78 Po 2+2+4, 77 Ch 2+6, 72 Ma 7, 76 Bh 6, 81 Bh 2+6, 69 Po 4, 72 Ash 2+4+4 · second most-asked question in the subject, and eight of the seventeen name XOR specifically · all four gates are worked in the companion

The perceptron

- A perceptron is a single artificial neuron with adjustable weights, a bias and a threshold activation, trained by an iterative rule that is guaranteed to converge when the classes are linearly separable. Rosenblatt, 1958.
- The difference from a McCulloch-Pitts neuron: the MP neuron's weights are chosen by hand, the perceptron's are *learnt*.
- Activation, with threshold θ :

$$f(\text{net}) = \begin{cases} 1 & \text{net} > \theta \\ 0 & -\theta \leq \text{net} \leq \theta \\ -1 & \text{net} < -\theta \end{cases}$$



- The perceptron learning rule. For each training pair, compute y ; if $y \neq t$, update, otherwise change nothing:

$$w_i(\text{new}) = w_i(\text{old}) + \alpha t x_i \quad b(\text{new}) = b(\text{old}) + \alpha t$$

- Weights change only when there is an error. That single sentence separates the perceptron rule from Hebbian learning (§7.4), which updates on every pattern.
- Convergence theorem: if a separating line exists, the rule finds one in a finite number of steps, whatever the starting weights.

Linear separability, and why it decides everything

- A single neuron computes $w_1x_1 + w_2x_2 + b$ and thresholds it. The set of inputs where that sum is zero is a straight line (a hyperplane in n dimensions). Everything on one side outputs 1, everything on the other outputs 0.
- So a perceptron can learn a function if and only if one straight line separates its 1s from its 0s. That is the whole of 81 Bh's "perceptron works as a linear classifier".
- Plot the four corners of the unit square:
 - AND – only (1, 1) is 1. One line cuts it off. ✓
 - OR – only (0, 0) is 0. One line cuts it off. ✓
 - XOR – the two 1s sit at opposite corners, with the two 0s at the other opposite corners. No straight line separates them. ✗
- Draw that square. Marks are given for the picture, and it takes ten seconds.

Limitations (76 Bh 6 asks precisely this)

- Learns only linearly separable classes.
- If the data is not separable the algorithm does not converge and its behaviour is not defined – it cycles.
- If it is separable there are usually many valid answers, and the rule gives no way to tell whether the one it found is a good one (no margin, no confidence). That is what SVMs later fixed.

The algebraic proof, which is what a "justify" question wants

- Suppose weights w_1, w_2 and bias b realise XOR with the rule output 1 iff $w_1x_1 + w_2x_2 + b > 0$. The four rows give:
 - (0, 0) → 0: $b \leq 0$ (i)
 - (0, 1) → 1: $w_2 + b > 0$ (ii)
 - (1, 0) → 1: $w_1 + b > 0$ (iii)
 - (1, 1) → 0: $w_1 + w_2 + b \leq 0$ (iv)
- Add (ii) and (iii): $w_1 + w_2 + 2b > 0$, so $w_1 + w_2 > -2b$.
- From (iv): $w_1 + w_2 \leq -b$. Together: $-2b < -b$, i.e. $b > 0$, which contradicts (i).
- Therefore no single perceptron can implement XOR. ■
- The fix: a multilayer perceptron. One hidden layer computes OR and NAND, and the output neuron ANDs them, so the two hidden units bend the decision boundary into two lines. Built and checked row by row in the companion.

7.3 Learning in a neural network: Hebbian, delta and back-propagation

What is the back-propagation algorithm? When do we need it? Explain with an example. 4 2+2+4 3+5

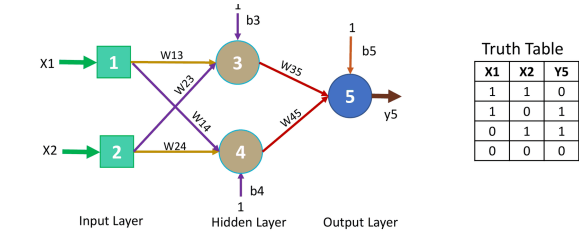
4+4 2+6 2+4+4

TOP 8/41 70 Bh 4+4, 72 Ash 2+4+4, 70 Ma 3+5, 80 Bh 2+2+4, 74 Bh 4, 79 Ch 2+6, 71 Ma 3+5, 72 Ash 3+5 · 71 Ma and 72 Ash attach it to machine vision (§7.9), so revise the two together · one full step is computed in the companion

The three learning rules, in one place

Rule	Update	Updates when
Hebb	$w_i += x_i t$	every pattern
Perceptron	$w_i += \alpha t x_i$	only on an error
Delta (LMS, Widrow-Hoff)	$w_i += \alpha (t - y_{in}) x_i$	every pattern, by how wrong it was

- The delta rule uses the *net input* y_{in} , not the thresholded output, so it measures the size of the error and not just its sign. That is what makes it differentiable, and therefore what back-propagation generalises.
- Adaline (Adaptive Linear Neuron, Widrow and Hoff 1960) is a single unit trained by the delta rule: identity activation while training, threshold afterwards. **On the syllabus, never yet examined**, so know it in four lines and no more.



Back-propagation

- Short for “backward propagation of errors”: a supervised algorithm that trains a **multilayer** network by gradient descent, pushing the output error backwards through the layers to get a gradient for every weight.
- When do we need it** (79 Ch 2): as soon as there is a **hidden layer**. A hidden unit has no target of its own, so the perceptron rule cannot say what its error is. Back-propagation manufactures one from the chain rule. No hidden layer, no XOR (§7.2), so no useful network.
- Why sigmoid:** $f(x) = \frac{1}{1+e^{-x}}$ has the derivative $f'(x) = f(x)(1 - f(x))$, so the forward pass already computed everything the backward pass needs. That is the only reason it is the default in every textbook example.

The back-propagation algorithm

- Six steps:**
 - 1.** Initialise all weights and biases to small random values, pick η .
 - 2. Forward pass:** layer by layer, compute $net_j = \sum w_{ij}x_i + b_j$ and $h_j = f(net_j)$ to the output y .
 - 3. Error at the output,** $E = \frac{1}{2}(y - t)^2$.
 - 4. Backward pass:** output delta $\delta_{out} = (y - t)y(1 - y)$; hidden delta $\delta_j = h_j(1 - h_j)\sum_k \delta_k w_{jk}$.
 - 5. Update every weight,** $w \leftarrow w - \eta \delta$ (input to that weight), and every bias, $b \leftarrow b - \eta \delta$.
 - 6.** Repeat over the training set until E is small enough or the epoch limit is reached.
- Problems to name:** slow; **gets stuck in local minima**; **vanishing gradient** through many sigmoid layers; sensitive to η and to the initial weights; can overfit.

Hebbian learning HOT 4/41 73 Bh 3+7, 81 Ba 2+6, 78 Ba 1+7, 78 Ch 6+3

- Hebb's postulate, 1949: *when cell A repeatedly takes part in firing cell B, the connection from A to B is strengthened*. Usually quoted as “**neurons that fire together wire together**”.
- Algorithm** (four steps, and this is the whole answer):
 - Step 0.** Set all weights and the bias to 0.
 - Step 1.** For a training pair (s, t) , set the inputs $x_i = s_i$.
 - Step 2.** Set the output to the target, $y = t$.
 - Step 3.** $w_i(new) = w_i(old) + x_i y$ and $b(new) = b(old) + y$.
 - Repeat for every pair. **One pass. No iteration, no learning rate, no error term.**
- Is Hebbian learning supervised?** (78 Ch 3) – **yes, as used here:** the update needs the target t , so the network is told the right answer. Hebb's biological postulate is unsupervised; the *Hebb net* for pattern association is not. Say both halves.
- It needs **bipolar** data. With binary inputs a 0 contributes nothing to any weight, so the AND net cannot be learnt at all – a real trap, and the reason every worked example uses ± 1 .

7.4 Hopfield network

What is a Hopfield network? Explain its features and all its steps with an example. 3 4 6 8 3+5

TOP 8/41 69 Bh 8, 75 Bh 8, 71 Bh 8, 80 Ash 3+5, 82 Bh 6, 80 Ch 4, 82 Ba 3, 71 Ma 3 · 82 Bh wants the weight matrix and a recall computed, which is the companion · 80 Ash and 71 Ma want it compared — with a feed-forward net and with Kohonen

- A Hopfield network is a single-layer, fully connected, recurrent network of binary units with symmetric weights, which settles from any starting state into the nearest stored pattern. Hopfield, 1982.
- **Structure**, and every word of it earns a mark:
 - One layer. Every unit is connected to **every other** unit, and each unit's output feeds back as an input.
 - Weights are **symmetric**, $w_{ij} = w_{ji}$, with **no self connection**, $w_{ii} = 0$.
 - Units are **binary** $\{0, 1\}$ or **bipolar** $\{-1, +1\}$.
 - **There is no separate output layer**: the same units take the input and hold the answer.
- **Hopfield Vs feed-forward** (80 Ash 5): recurrent Vs one-way; symmetric weights Vs arbitrary; one layer Vs many; associative memory Vs function approximation; converges to a stored pattern Vs computes an output in one pass; trained in **one shot** Vs trained by many epochs of back-propagation.

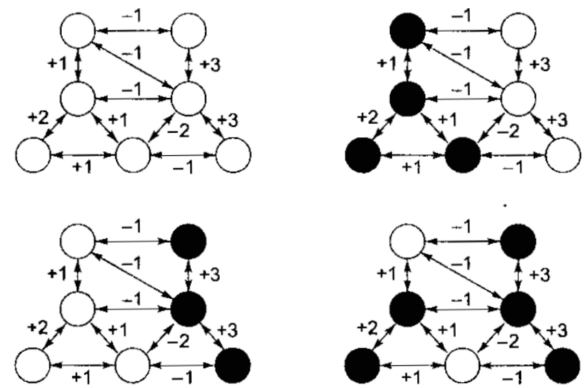


Fig. 18.2 The Four Stable States of a Particular Hopfield Net

Four stable states of one small net. Supply any part of one and the network falls into it.

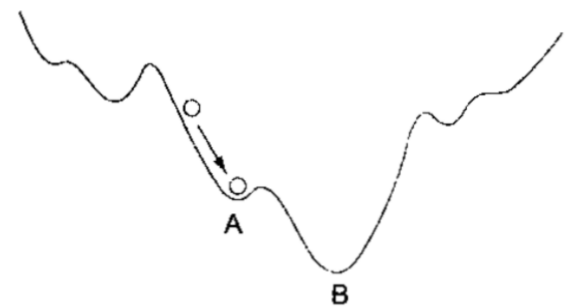


Fig. 18.3 A Hopfield Net as a Model of Content-Addressable Memory

Every update walks downhill on this surface, so the network always stops somewhere. The valleys are the stored patterns.

- **The four features to list** (71 Bh asks for them by name):
 - **Content-addressable memory** – retrieve a whole pattern by supplying *part* of it. This is the headline property.
 - **Distributed representation** – a pattern is stored in all the weights at once, not in one cell.
 - **Asynchronous control** – units update one at a time, in any order, with no central clock.
 - **Fault tolerance** – damage a few weights and recall degrades gracefully rather than failing.
- **Energy.** Each state has an energy

$$E = -\frac{1}{2} \sum_{i \neq j} w_{ij} s_i s_j - \sum_i \theta_i s_i$$

and every update either lowers it or leaves it alone. So the network **always converges**, to a local minimum of E . The stored patterns are those minima.

- **Capacity:** about $p \leq 0.15N$ patterns for N units. Store more and the patterns interfere; the net then settles into **spurious states** that were never taught.

The algorithm

- **Training** (one pass, no iteration). For P bipolar patterns:

$$w_{ij} = \sum_{p=1}^P s_i^{(p)} s_j^{(p)} \quad (i \neq j), \quad w_{ii} = 0$$

i.e. sum the outer products and zero the diagonal.

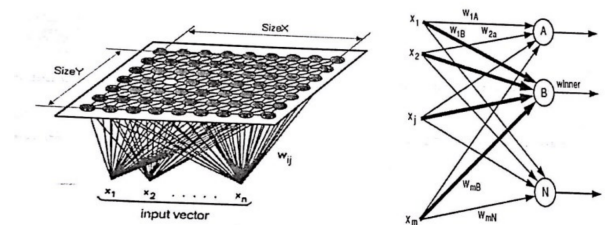
- **Recall / testing**, given a noisy input x :
 - **1.** Set the state $y = x$.
 - **2.** Pick one unit i at random and compute $y_{in,i} = x_i + \sum_j y_j w_{ji}$.
 - **3.** Apply the threshold: $y_i = 1$ if $y_{in,i} > \theta_i$, unchanged if equal, 0 (or -1) if less.
 - **4. Broadcast the new y_i immediately** – the next unit uses it. This is what *asynchronous* means, and updating all units at once is the commonest mistake.
 - **5.** Repeat until no unit changes. The state is then stable.
- **Worked in full in the companion:** three patterns stored, the weight matrix built, and a corrupted pattern recalled unit by unit.

7.5 Kohonen network and the self-organizing map

What is a self-organizing map? Explain all the steps involved with an example. 3 4 5 2+5

TOP 9/41 74 Bh 2+5, 80 Bh 4, 79 Bh 4, 75 Bh 4, 74 Ma 4, 72 Ash 4, 79 Jth 5, 81 Bh 3, 71 Ma 3 · seven of the nine ask it as a short note, so the answer is a definition, the algorithm as five steps, and one application · 71 Ma wants it compared with Hopfield

- A self-organizing map (Kohonen network) is an unsupervised, competitive neural network that maps high-dimensional input onto a low-dimensional grid of neurons while preserving the topology of the data. Kohonen, 1982.
- **Structure:** an input layer, and **a grid of output neurons** (usually 2-D). Fully connected between them and **nothing else** – no hidden layer, no bias, no activation function.
- **Competitive learning:** for each input, exactly one output neuron **wins**, the one whose weight vector is nearest the input. It and its *neighbours* are pulled towards that input; everyone else is left alone.
- **Topology preserving** is the property to name: inputs that are similar end up firing neurons that are close together on the grid. That is what makes an SOM a visualisation tool and not just a clustering algorithm.
- **Used for:** clustering, dimensionality reduction, data visualisation (the poverty map is the classic), feature extraction, image compression.



The grid of neurons, and the same net drawn as connections from the input vector to every output neuron with one marked winner.

Hopfield Vs Kohonen (71 Ma 3)

	Hopfield	Kohonen
Learning	supervised (patterns given)	unsupervised
Structure	one layer, fully recurrent	input layer → output grid, feed-forward
Weights	symmetric, $w_{ii} = 0$	one vector per output neuron
Rule	Hebbian outer product, one shot	competitive, winner takes most
Output	a stored pattern	the winning neuron's position
Used for	associative memory, optimization	clustering, visualisation

– ✓ Easy to interpret; handles large, complex data. × Computationally expensive; the grid size and the initial weights are guesses; it assumes nearby points should behave alike.

The algorithm, five steps

- **Step 1. Initialise** every neuron's weight vector w_j randomly, and set the learning rate α and the neighbourhood radius σ .
- **Step 2. Present** an input vector x .
- **Step 3. Find the winner** (the Best Matching Unit), the neuron whose weight vector is nearest:

$$D(j) = \sum_i (x_i - w_{ij})^2, \quad j^* = \arg \min_j D(j)$$

the *squared* Euclidean distance is enough — the square root does not change which one is smallest, and skipping it saves time in an exam.

- **Step 4. Update the winner and its neighbours**, moving them towards the input:

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + \alpha (x_i - w_{ij}(\text{old}))$$

for every j inside the neighbourhood of j^* .

- **Step 5. Shrink** α and σ , and repeat from step 2 until they reach their floor or the map stops moving.
- **Two phases worth naming:** an early **ordering** phase with a large radius that lays the map out, and a long **convergence** phase with a small radius that fine-tunes it.

7.6 Expert systems: what they are, and the architecture

What is an expert system? Explain its general architecture with a block diagram. 4 6 8 2+4 1+7 2+5 3+5

TOP 17/41 69 Po 8, 71 Ma 2+4, 76 Ba 1+7, 71 Bh 8, 80 Ba 2+5, 81 Ba 3+5, 77 Ch 6+2, 78 Ch 1+4+3, 76 Bh 5+2+2, 70 Ma 2+6, 72 Ma 3+6, 73 Ma 3+5, 75 Ba 2+6, 80 Ch 6, 82 Ba 8, 71 Bh 2+6, 74 Bh 2+5 · **joint most-asked question in the subject** · three papers dress it as a scenario — *disease diagnosis* (80 Ch), *a disaster-prone area of Nepal* (82 Ba), *create a scenario where we require one* (79 Ch) — and want the same block diagram with the domain renamed

Definition

- An expert system is a computer program that simulates the decision-making of a human expert in one narrow domain, using a body of high-quality specific knowledge rather than a general algorithm.
- **Narrow is the operative word.** An expert system is expert at one thing and useless at everything else.
- Knowledge comes from **human specialists**, and also from texts, journals and databases.

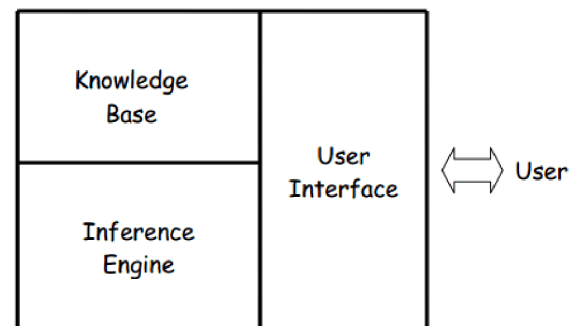


Fig: Block Diagram of expert system

Draw this. Four boxes and one double-headed arrow to the user is a full answer to the block-diagram half of every one of the seventeen papers.

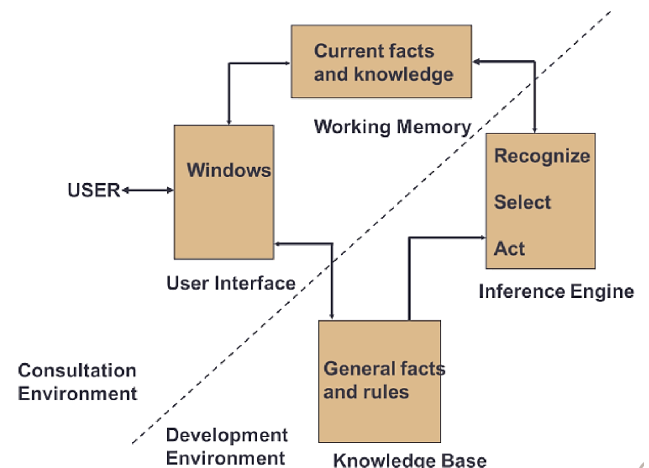
- **Expert system shell** (82 Ka 2): an expert system with the **knowledge base emptied out** – inference engine, user interface and explanation facility, ready for a new domain's rules. It is what turns building an expert system into writing rules instead of writing code. Eg EMYCIN, the shell left when MYCIN's medical rules were removed.
- **When is a problem suited to one** (82 Ka 3): the domain is **narrow and well bounded**; genuine **human experts exist** and can explain their reasoning; the task is **symbolic and heuristic**, not numerical; it takes an expert minutes to hours, not seconds or months; and no ordinary algorithm solves it.

How the inference engine works (81 Ba 5, 82 Bh 5)

- It applies one rule of logic, **modus ponens**: if $A \rightarrow B$ is in the knowledge base and A is in working memory, add B .
- **Two directions**, both from Chapter 4:
 - **Forward chaining (data driven)** – start from the facts, fire whatever matches, add the conclusions, repeat. Use it when there are few facts and many possible conclusions. *Monitoring, planning, design.*
 - **Backward chaining (goal driven)** – start from a hypothesis and look for rules that would prove it, asking the user for what is missing. Use it when there are few plausible conclusions. *Diagnosis.* MYCIN works this way, and it is why the system asks you questions.
- **The recognize–select–act cycle**, which is the answer if the question says *how is inference performed*:
 - **1. Match** – compare every rule's condition against working memory, producing a **conflict set** of rules that could fire.
 - **2. Conflict resolution** – choose *one* of them, by specificity, recency, rule priority or a **meta-rule**.
 - **3. Act** – fire it, updating working memory. Repeat until the goal is reached or nothing matches.
- **Rule types to name**: **relationship** (*IF the battery is dead THEN the car will not start*), **recommendation** (*... THEN take a cab*), **directive** (*... AND the fuel system is ok THEN check the electrical system*), **heuristic**, and **meta-rules**, which are rules about which rules to use.

The components

- **Knowledge base** – the domain knowledge, almost always as **IF–THEN production rules**: *IF the battery is dead THEN the car will not start*. This is what makes it an expert system rather than a program.
- **Working memory** – the facts about *this* case, built up during the consultation. Sometimes called the database or the fact base.
- **Inference engine** – matches rules against working memory and fires them. Also called the control structure or rule interpreter.
- **User interface** – two-way: it takes the user's queries and answers, and it presents conclusions.
- **Explanation facility** – answers *why are you asking* and *how did you conclude that*. **Name it: it is the one component most answers forget**, and it is what makes an expert system acceptable to a professional.
- **Knowledge acquisition subsystem** – how new rules get in (§7.7).



The fuller picture, worth drawing when the question is worth 6 or more: the dashed line splits the consultation environment the end user sees from the development environment the knowledge engineer works in.

Two real systems to name

- **MYCIN** (Stanford, 1970s) – diagnoses blood infections and recommends antibiotics from about **500 production rules**. Uses **backward chaining**, and will explain its reasoning on request. Matched human specialists.
- **DENDRAL** (Stanford, late 1960s) – the **first** expert system. Infers molecular structure from mass spectra using chemists' heuristics. **Forward chaining**.
- Others worth one word each: **XCON** (configuring VAX computers), **PROSPECTOR** (mineral exploration), **CADUCEUS** (internal medicine).

7.7 Expert system development, and knowledge acquisition

Explain the stages of expert system development, with an example. 6 8 2+5 2+6 4+4

TOP 8/41 69 Bh 8, 70 Bh 4+4, 73 Bh 2+6, 75 Ba 2+6, 79 Ch 2+6, 80 Ch 6, 82 Ba 8, 80 Ba 2+5 · always paired with the block diagram, so answer both halves · 80 Ch casts you as the knowledge engineer for a disease-diagnosis system, 82 Ba in a disaster-prone area of Nepal

The six stages

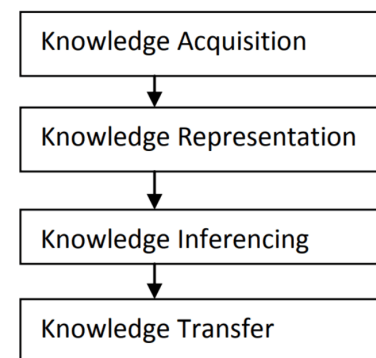
- **1. Identify the problem domain.** Is the problem suitable at all? Are there real experts, and will they cooperate? Is it cost-effective?
- **2. Design the system.** Choose the technology and the shell; decide how the domain knowledge will be *represented*; settle the integration with existing databases.
- **3. Develop the prototype.** The **knowledge engineer** acquires the domain knowledge from the expert and encodes it as IF-THEN rules. This is the long stage.
- **4. Test and refine.** The knowledge engineer runs sample cases; end users try the prototype; rules are corrected and added. **Iterate 3 and 4 until it performs like the expert.**
- **5. Develop and complete.** Integrate with the real environment, document it, train the users.
- **6. Maintain.** Keep the knowledge base current as the domain changes, and keep the interfaces working as the surrounding systems evolve.

Answering the scenario version

- **Name the domain, the expert and three real rules.** For 80 Ch's disease diagnosis: expert = a physician; rules like *IF fever > 102 °F AND rash AND joint pain THEN suspect dengue, certainty 0.8*; working memory = this patient's symptoms; backward chaining, because you are testing a hypothesis.
- Then draw the block diagram with the boxes *relabelled for that domain*, and walk the six stages in one line each. That is a full 8 marks.

Knowledge acquisition PIN 2/41 71 Bh 2+6, 74 Bh 2+5

- Knowledge acquisition is the process of extracting knowledge from human experts and other sources and encoding it into the knowledge base.
- **It is the bottleneck of the whole field.** Experts know more than they can say, disagree with each other, and are expensive; and what they do say has to be turned into rules by someone who is not an expert.
- **The methods:** structured and unstructured interviews; observation of the expert working; questionnaires; protocol analysis (the expert thinks aloud); case studies; and automatic acquisition, i.e. machine learning from records (Ch 6), which is what removes the bottleneck when data exists.
- **Sources:** the expert (primary), then secondary and tertiary experts, then literature – reports, guidelines, books, manuals – then end users.
- **The four kinds of knowledge to name:** declarative (facts), procedural (how to do it), episodic (remembered cases), meta-knowledge (knowledge about the knowledge).
- **Example to give:** interviewing a cardiologist about how she reads an ECG, recording each judgement as a rule, then replaying old cases to see where the rules disagree with her.



A second, coarser framing of the same thing, and the one BA Sir's deck gives. It is not a rival to the six stages: acquisition and representation are stage 3, inferencing is stage 4, transfer is stage 5.

- **Either list is accepted.** Give the six stages, and draw this beside them if the question says "with a diagram" – it is the only flow in the answer, the block diagram in §7.6 being a *structure*.

7.8 Expert systems: characteristics, advantages, limits, and Vs human experts

What are the characteristics of an expert system? Compare it with a human expert. Give its advantages and disadvantages. 3 4 7 8 2+2+3

TOP 11/41 72 Ash 8, 81 Ch 4+3, 80 Ash 8, 81 Ash 4, 79 Bh 4, 78 Ch 1+4+3, 76 Bh 5+2+2, 77 Ch 6+2, 82 Ka 2+2+3, 80 Ba 2+5, 82 Bh 2+5 · 80 Ash 8 asks for the comparison alone, with practical examples for each point, so the table below is that answer

Pro Expert system	Human expert	Expert system
	Perishable(decay)	Permanent
	Slow processing	Fast processing
	Unpredictable	Consistent
	Slow reproduction	Quick replication
	Expensive	Affordable
Pro Human	Broad focus	Narrow focus
	Inspiration	Lacks creative ability
	Adaptive needs	Instruction
	Common sense	Machine knowledge

Which side wins each point. The top block is where the machine is better, the bottom block where the person is.

Characteristics (80 Ba 2, 82 Ka 2)

- **High performance** in its narrow domain, at expert level.
- **Understandable** – takes input and gives output in the user's language.
- **Reliable and highly responsive.**
- **Explains itself**, and separates **knowledge from the program that uses it**, so the knowledge can be edited without touching the code.
- Uses **symbolic** representation and **heuristics**, not an algorithm.
- Handles **uncertainty**, with certainty factors or probabilities (Ch 4).

Advantages

- **Permanent**: it does not retire, forget, or die. A human expert's knowledge leaves with them.
- **Reproducible** – copy it to a hundred sites at no extra cost, so scarce expertise reaches places that have none. This is the argument for 82 Ba's disaster scenario.
- **Consistent**: same input, same answer, every time. A tired human is not.
- **Fast**, available **24 hours**, and **cheaper** than the expert.
- Works in **hazardous environments**; keeps a full audit trail; and is a **training tool**, since it explains its reasoning.

Disadvantages and limitations

- **No common sense**. It has knowledge, not understanding, and will give a confidently absurd answer just outside its domain.
- **Narrow**: cannot transfer anything it knows to a neighbouring problem.
- **Cannot learn from experience** on its own – every new rule has to be put in by a knowledge engineer.
- No **creativity** or intuition, and no ability to recognise that the case in front of it is unusual.
- **Expensive and slow to build**, because of the knowledge-acquisition bottleneck; and **only as good as the expert it copied**, errors included.
- Cannot take moral responsibility for a decision, which is why medical expert systems advise rather than decide.

Declarative Vs procedural knowledge HOT 4/41 70 Ma 2+6

, 72 Ma 3+6 , 73 Ma 3+5 , 72 Ma 6

all four ask it *with* the expert system architecture, so the two are one answer.

	Declarative	Procedural
Is	knowing what	knowing how
Content	facts about objects, events, situations	the steps to carry a task out
Form	statements: logic, semantic nets, frames (Ch 4, Ch 5)	rules, programs, procedures
Stored	separately from how it is used	inseparably from the code using it
Changed	easily, one fact at a time	only by rewriting the procedure
Reused	by any inference method	only for its own purpose
Efficiency	lower: control must be worked out at run time	higher: control is built in
Example	<i>Kathmandu is the capital of Nepal;</i> <code>dog(Tommy)</code>	<i>how to ride a bicycle;</i> a sorting routine

In an expert system, both are present (this is the half the papers actually mark)

- The **knowledge base** is **declarative**: it states what is true in the domain and says nothing about how to use it.
- The **inference engine** is **procedural**: it knows how to search and fire rules, and nothing about the domain.
- **That separation is the whole design**, and the reason a shell exists at all: swap the declarative half and the same procedural half serves a new domain.
- An IF-THEN rule sits on the line, which is worth a sentence: it *states* a relationship, so it reads declaratively, but firing it *does* something, so it behaves procedurally. That is why production rules are the standard expert system representation.

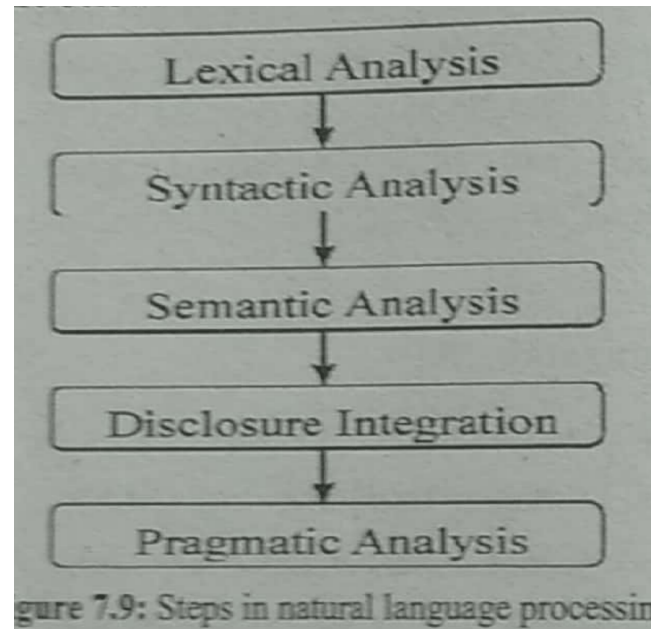
7.9 Natural language processing: NLU, NLG and the five steps

What is NLP? Explain the different steps involved, with a block diagram and examples. 5 6 7 8 9 2+6
3+5

TOP 23/41 69 Bh 6, 69 Po 8, 73 Ma 2+4, 79 Ch 3+5, 72 Ash 2+6, 75 Ba 8, 72 Ma 8, 73 Bh 7, 73 Bh 2+7, 74 Ma 9, 74 Bh 2+6, 82 Ba 2+6, 79 Jth 1+6, 82 Ka 2+5, 81 Ch 2+5, 77 Ch 2+6, 78 Po 3+5, 79 Ash 5+2, 75 Bh 6+2+2, 76 Bh 5+4, 78 Ba 2+6, 78 Ch 2+4+2, 80 Ch 2+6 · the most-asked question in the whole subject, in 23 of the 41 papers, and it has been set at least once in every year from 2069 to 2082 · eight of the twenty-three also want NLU and NLG defined · 80 Ch asks it as *build a Nepali chatbot*

What NLP is

- Natural language processing is the branch of AI that analyses written or spoken human language and converts it into a representation a computer can use, and back again.
- Two halves, and the papers ask for both by name:
 - NLU, natural language understanding – text in, meaning out. Maps the input to a useful representation: entities, concepts, keywords, relations, sentiment, semantic roles. **This is the hard half.**
 - NLG, natural language generation – meaning in, text out. Its own three stages: **text planning** (what to say), **sentence planning** (how to phrase and combine it), **text realisation** (produce grammatical output).
- Input is text or speech, and **the quality of the input bounds everything downstream**: a bad transcription cannot be recovered by a good parser.
- **Applications** to name: machine translation, chatbots and virtual assistants, sentiment analysis, information retrieval and search, spam filtering, text summarisation, question answering, speech recognition, spelling and grammar correction, optical character recognition.



Draw these five boxes in this order. It is the single most valuable minute of drawing in the subject. (The published figure prints *Disclosure Integration*; the term is **Discourse Integration** – a misprint in the book, not a different step.)

The five steps, each with the example to give

- **1. Lexical / morphological analysis.** Break the text into paragraphs, sentences and **tokens**, and analyse the structure of each word.
 - “I love NLP, it’s fascinating!” →
[I] [love] [NLP] [,] [it’s] [fascinating] [!]
 - Includes **stemming** (rule-based chopping: *closed, closing, closer* → *clos*) and **lemmatization** (dictionary-based: *better* → *good*). Name the difference; it is a reliable extra mark.
 - Morphology: *unhappily* = *un-* + *happy* + *-ly*.
- **2. Syntactic analysis (parsing).** Check the words against the grammar and build a **parse tree** showing how they combine.
 - “The school goes to boy” is **rejected here** – every word is a real word, but the arrangement is not English.
 - Includes **part-of-speech tagging**: *book* is a noun in *the book on the table* and a verb in *to book a flight*.
 - A worked parse tree is in the companion.

The four levels, and the ambiguity at each TOP 8/41 71 Ma 2+2+2+2

, 71 Bh 8 , 72 Ma 6 , 78 Ba 2+6 , 72 Ash 1+7 , 82 Bh 3+4 , 80 Bh 3+5 , 80 Ba 2+5

eight papers ask for the *levels* rather than the steps, usually with “give an example of ambiguity at each”. Same material, different cut: give one example per level and you have the marks.

Level	Concerns	An ambiguity
Phonetic	sounds → words	“ice cream” against “I scream”; “grey day” against “grade A”
Lexical	which sense of a word	<i>bank</i> : riverbank or institution
Syntactic	which grammatical structure	“I saw the man with a telescope”: who has the telescope?
Semantic	what the sentence means	“The chicken is ready to eat”: is it the eater or the meal?
Pragmatic	what the speaker intended	“Can you open the window?”: request, not a question

Steps 3 to 5

- **3. Semantic analysis.** Turn the parse into meaning.
 - **Word sense disambiguation**: *bank* the riverbank or *bank* the institution; *Mary had a bat in her office*.
 - Sentence-level meaning: “Colourless green ideas sleep furiously” is perfect syntax and **rejected here**, which is the clean example of what step 3 does that step 2 cannot.
- **4. Discourse integration.** The meaning of a sentence depends on the sentences around it.
 - “The thieves stole the paintings. They were subsequently sold / caught / found.” **One word changes who “they” is.**
 - “John wanted it” means nothing without the previous sentence.
- **5. Pragmatic analysis.** What was actually *meant*, using real-world knowledge.
 - “Can you pass the salt?” is a **request**, not a question about your arm. Answering yes is grammatically perfect and pragmatically wrong.
 - Covers sarcasm, idiom and implied meaning.
- **Phonetic is the level every deck skips**, and six papers list it. It is the level at which *sound* is turned into *words*, and its ambiguity is that one stream of sound splits into words in more than one way.
- **Syntactic and semantic analysis compared** (80 Bh, 80 Ba, 82 Bh ask exactly this): syntax asks *is this arrangement legal, and what is its structure* and answers with a parse tree; semantics asks *what does that structure mean* and answers with a meaning representation. A sentence can pass one and fail the other, in both directions.

7.10 Issues and challenges in NLP, and machine translation

What are the problems associated with NLP? Why is it hard? 2 3 5 8 1+7

TOP 13/41 70 Bh 1+7, 70 Ma 2+6, 78 Ch 2+4+2, 82 Ka 2+5, 81 Ch 2+5, 77 Ch 2+6, 78 Po 3+5, 79 Ash 5+2, 75 Bh 6+2+2, 79 Bh 2+5, 81 Ash 8, 80 Bh 3+5, 80 Ba 2+5 · almost always the 2 or 3 opener to the steps question · 81 Ash 8 and 80 Ch 2+6 set it as a Nepali-language problem and want the issues related to Nepali

Why natural language is hard

- **Ambiguity at every level** – §7.9's table. This is the root cause and should be your first sentence.
- **Context dependence**: “Where’s the water?” means something different in a chemistry lab, when you are thirsty, and under a leaking roof.
- **Many ways to say one thing**: “Ram’s birthday is October 11” and “Ram was born on October 11”.
- **The language will not stay still**: new words, new senses, slang. No lexicon is ever complete.
- **Hidden and figurative meaning**: idiom, metaphor, sarcasm.
- **Pronouns and reference**: “Ravi went to the supermarket. He found his favourite coffee. He paid for it and left.” What is it?
- **Ungrammatical and incomplete input**: “He rice eats.” People still understand it; a parser does not.
- **Ellipsis and conjunction**: “Ram and Hari went to a restaurant. Ram had coffee, Hari tea.” – the second verb is simply missing.
- **Speech adds its own**: accents, homophones, and **no spaces between spoken words**.

Machine translation (70 Bh 1+7)

- **Machine translation is the automatic conversion of text or speech from one natural language into another, preserving meaning.**
- **Three approaches**, in historical order:
 - **Rule based** – dictionaries plus hand-written grammar rules. Accurate where the rules cover, brittle everywhere else.
 - **Statistical** – learn word and phrase probabilities from a **parallel corpus**. Needs vast aligned data.
 - **Neural** – an encoder-decoder network trained end to end. The current standard, and the reason translation improved suddenly after 2016.
- **Its challenges**: word-sense ambiguity; idioms that must not be translated literally; different **word order** (English SVO, Nepali SOV); grammatical gender and honorifics with no counterpart in the other language; named entities and untranslatable culture-specific words; and no parallel corpus for low-resource pairs.

The Nepali version of the answer (81 Ash 8, 80 Ch 6)

- **Low-resource language**: little annotated text, no large treebank, few parallel corpora. Everything below follows from this.
- **Devanagari script**: transliteration and code-mixing (Nepali typed in Roman letters), inconsistent Unicode encoding, no standard spelling for many loanwords.
- **Morphologically rich**: heavy inflection and agglutination, so one root yields dozens of surface forms and a lexicon lookup fails.
- **Honorifics and levels of respect** (*timi / tapai / hajur*) change the verb, and carry meaning a translation must preserve.
- **Free word order**, and **subject-object-verb**, so English-trained parsers transfer badly.
- **Dialects** and heavy loan vocabulary from English and Hindi.
- **Mitigation, which is the half that carries the marks**: build and release an annotated corpus; use transfer learning from a multilingual model rather than training from scratch; augment data by back-translation; write a morphological analyser for the inflections; standardise the encoding; keep a human in the loop for evaluation.

Tools NLP needs (the “knowledge bases required”, 80 Ch 2)

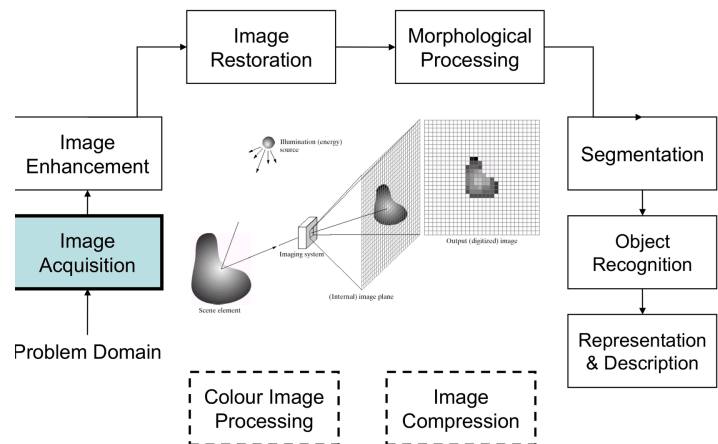
- **Lexicon / dictionary** – the vocabulary, with meanings, pronunciations and syntactic categories.
- **Morphological analyser** – splits words into prefix, root and suffix and derives their grammatical features.
- **Grammar** – the rules that say which arrangements are legal, used by the parser.
- **Semantic and world knowledge** – an ontology or knowledge base (Ch 5) that says what the words refer to and what is plausible.
- **Annotated corpus** – tagged and parsed text to train and to test on.
- **Discourse and pragmatic model** – to resolve pronouns and intent.

7.11 Machine vision

What is machine vision? Why is it important in AI? List its steps and applications. 3 4 2+4 2+5 2+6 3+5
1+5+2

TOP 12/41 71 Ma 3+5, 72 Ash 3+5, 80 Ash 2+6, 81 Ash 2+4, 81 Bh 2+5, 79 Ash 1+5+2, 69 Bh 4, 80 Ch 4, 80 Ba 3, 82 Ba 3, 79 Bh 2+5, 70 Bh 2+6 · two papers pair it with back-propagation (71 Ma, 72 Ash) and one with GA (70 Bh), so it is often the other half of a neural-network question · 80 Ash wants a scenario and the steps to solve it

- Machine vision is the field of AI that builds a model of the real world from images – recovering useful information about a three-dimensional scene from its two-dimensional projections.
- The hard part in one line:** the world is 3-D and the image is 2-D, so information has been thrown away and must be put back using knowledge about the objects and the projection geometry.
- An image is a 2-D array of intensity values. Each element is a pixel. Grey scale $f = f(x, y)$; colour is three such functions, $\{R(x, y), G(x, y), B(x, y)\}$.



anitha Acharya (aolunthazas@gmail.com)

Why it matters to AI (71 Ma 3, 79 Bh 5)

- Vision is how an agent perceives**, and Chapter 1's agent model begins with sensors. Without it an agent is limited to whatever a human types in.
- It is the only practical channel for **unstructured** information about the physical world: no one can hand-code a description of every scene.
- It is what makes autonomy possible – robots, vehicles, inspection – and it is where neural networks work best, which is why **back-propagation** (§7.3) and machine vision are asked together.

The stages, in order

- 1. Image acquisition** – capture the scene with a camera or sensor and digitise it. Lighting is a design decision, not an accident.
- 2. Image enhancement** – make features easier to see: contrast stretching, sharpening, noise filtering, edge detection.
- 3. Image restoration** – undo a *known* degradation (blur, sensor noise) using a model of how it happened. Enhancement is subjective, restoration is objective; say so.
- 4. Morphological processing** – operate on shape: erosion, dilation, opening, closing, to clean up the regions.
- 5. Segmentation** – **partition the image into objects and background**, grouping pixels that share intensity, colour, texture or motion. The hardest step, and the one everything after depends on.
- 6. Representation and description** – turn each region into **features**: boundary or area, size, shape, moments, texture.
- 7. Object recognition** – assign a label to each region by **pattern recognition**: statistical (feature vectors into a classifier) or structural (decompose into primitives).
- 8. Interpretation** – say what the scene *means*: how many, where, moving how, and what to do about it.
- Colour processing** and **image compression** sit beside the pipeline rather than in it; the block diagram draws them as separate boxes and so should you.

Applications

- **Industry:** automated inspection on a production line, reading part numbers, robot guidance and pick-and-place, barcode and label verification.
- **Medicine:** X-ray, CT and MRI analysis, tumour detection, cell counting.
- **Transport:** self-driving cars, lane and sign detection, number-plate recognition, traffic monitoring.
- **Security:** face recognition, fingerprint and iris biometrics, surveillance, intrusion detection.
- **Agriculture:** crop and disease monitoring, fruit grading, yield estimation.
- **Documents:** OCR, handwriting recognition, cheque processing.
- **Remote sensing and astronomy; AR and VR.**

Answering the scenario version (80 Ash 6, 81 Ash 4)

- **Pick a concrete scenario and walk the eight stages through it.** That is the answer; a generic list of stages is half marks.
- Worked example – **sorting apples on a conveyor:**
 - **Acquire:** overhead camera, diffuse lighting to kill shadows.
 - **Enhance:** median filter to remove sensor speckle.
 - **Restore:** deblur for the belt's motion, which is known and constant.
 - **Morphology:** close small gaps so each apple is one solid region.
 - **Segment:** threshold on colour to separate fruit from belt.
 - **Describe:** for each region take area, roundness, mean hue, and the fraction of dark pixels.
 - **Recognise:** a classifier sorts each into grade A, grade B or reject.
 - **Interpret and act:** fire the diverter arm for the right bin.
- Say what would break it – overlapping apples defeat segmentation, changing daylight defeats the colour threshold – and how you would fix it. That sentence is what separates a 6 from a 4.

7.12 Every Chapter 7 question, grouped by the answer it wants

Bold = regular exam, plain = back. *Monospace* = CT 710 (BEI), *italic* = CT 78506 (BEX), upright = CT 653 (BCT).

- **Steps of NLP 23 papers** → §7.9. Wordings, all one answer: *explain the steps* (69 Po, 73 Ma, 79 Ch, 75 Ba, 72 Ma, 73 Bh); *define NLU and NLG, then the steps* (73 Bh, 74 Ma, 74 Bh, 82 Ba, 79 Jth); *the problems, then the steps* (82 Ka, 81 Ch, 77 Ch, 78 Po); *why is NLP hard* (79 Ash); *steps and a parse tree for "A girl has a name"* (76 Bh, worked in the companion); *a Nepali chatbot* (80 Ch).
- **NLP levels and their ambiguities 8 papers** → §7.9's second half (71 Ma, 71 Bh, 72 Ma, 78 Ba, 72 Ash, 82 Bh, 80 Bh, 80 Ba). Phonetic is the level to make sure you can define.
- **NLP issues, challenges, machine translation 13 papers** → §7.10, including both Nepali-language questions (81 Ash, 80 Ch).
- **Perceptron and the logic gates 17 papers** → §7.2, all worked in the companion. **OR** (72 Ma); **AND** (73 Bh, 78 Po, 77 Ch); **XOR** (73 Ma, 74 Ma, 74 Bh, 82 Bh, 79 Ash, 77 Ch); *any logic gate* (79 Jth); *linear separability* (76 Bh, 81 Bh); *MLP* (72 Ma, 77 Ch); short note (69 Po).
- **Expert system architecture 17 papers** → §7.6. **Development stages 8 papers** → §7.7. **Advantages, limits, Vs human experts 11 papers** → §7.8. **Declarative Vs procedural 4 papers** → §7.8. **Knowledge acquisition 2 papers** → §7.7.
- **Machine vision 12 papers** → §7.11.
- **Kohonen / SOM 9 papers** → §7.5. **Hopfield 8 papers** → §7.4, with the weight matrix and recall worked in the companion for 82 Bh.
- **Back-propagation 8 papers** → §7.3, one full step worked in the companion. **Hebbian learning 4 papers** → §7.3, the AND net worked in the companion.
- **The neuron model, McCulloch-Pitts, activation function 8 papers** → §7.1.
- **Forward propagation through the printed network** (PIN 2/41 82 Ka 8, 81 Ch 8) – the only genuine numerical in the chapter, and **the same question printed twice with the same figure**. Worked in the companion, including why the ReLU layer's weights cannot be updated at all for the given input.
- **Short notes that live here:** *learning in a neural network* (69 Po, 74 Bh, 78 Ba, 77 Ch, 81 Ash), *learning rate and learning based on statistics* (72 Ma), *human brain Vs neural network* (69 Bh).
- **Adaline** – on the syllabus, **never examined in 41 papers**. Four lines in §7.3 and no more.
- **Pattern recognition** – in the textbook's §7.4 but not on the CT 710 syllabus and never asked on its own. It appears here only as step 7 of the machine-vision pipeline.

Source notes

- Spine: BA Sir *CH-07 Applications of AI* (the fullest single source in *Notes* for this chapter), PS Sir 7.1 *Neural Network*, 7.2 *Expert Systems* and 7.3 *NLP and Machine Vision*, and *Insights on Artificial Intelligence* ch 7 for the worked method.
- **One correction carried into the figure caption:** the published NLP figure prints *Disclosure Integration*; the step is **Discourse Integration**. The same misprint runs through several decks.
- **Phonetic-level ambiguity is** verified/added. Six papers ask for the four levels of analysis and no deck in *Notes* explains the phonetic one; the examples here were researched and written.
- **The Nepali-language answers are** verified/added for 81 Ash and 80 Ch: no source covers them, and both questions are worth 6 or 8 marks.
- **Hebbian learning** is not on the syllabus under any name but is asked in four papers, so it is written in full in §7.3 and worked in the companion.

Abbreviations used in these notes

Only these are ever shortened. Anything not on this list is written out in full.

Short	Means	Short	Means
AI	artificial intelligence	ANI	artificial narrow intelligence
AGI	artificial general intelligence	ASI	artificial super intelligence
KR	knowledge representation	KRR	knowledge representation and reasoning
PEAS	performance measure, environment, actuators, sensors	GPS	General Problem Solver
NLP	natural language processing	NLU	natural language understanding
NLG	natural language generation	CAPTCHA	completely automated public Turing test to tell computers and humans apart
ML	machine learning	ANN, NN	artificial neural network
MLP	multilayer perceptron	BP	back-propagation
SOM	self-organizing map	GA	genetic algorithm
FIS	fuzzy inference system	ID3	Iterative Dichotomiser 3
CSP	constraint satisfaction problem	DFS	depth first search
BFS	breadth first search	DLS	depth limited search
IDS	iterative deepening search	UCS	uniform cost search
CNF	conjunctive normal form	WFF	well-formed formula
FOL	first-order logic	FOPL	first-order predicate logic
RRS	resolution refutation system	BBN	Bayesian belief network
Short	Means	Short	Means
ES	expert system	KB	knowledge base
CD	conceptual dependency	LISP	list processing (the language)
MOP	memory organization packet	isa	the class-to-class link of a semantic net (<i>instance</i> is the individual-to-class one)
attr(s)	attribute(s)	repo	repository
freq	frequency	config	configuration
info	information	no.	number
Eg	example	Vs	compare / differentiate
w/, w/o	with, without	+Fig	draw the figure
→	leads to, therefore	=	is, equals

Reading the year tags and chips

Exam-paper month codes: Ba = Baishakh · Jth = Jestha · Shr = Shrawan · Bh = Bhadra · Ash = Ashwin · Ka = Kartik · Po = Poush · Ma = Magh · Ch = Chaitra. Years are Bikram Sambat, so 80 Ch is 2080 Chaitra.

Bold year tags are Regular exams, plain are Back exams. A header printing “Regular / Back” in one box counts as Regular.

Three course codes, three faces. Plain = **CT 653** (BCT, III-II, 26 papers) · `monospace` = CT 710 (BEI, IV-I, 7 papers) · *italic* = CT 78506 (BEX, IV-II, 8 papers). **The same year code names different papers under different codes:** 77 Ch and 77 Ch are two different exams, and only the font tells them apart.

Tier chips over all 41 papers, 2069–2082 BS: **TOP /41** = topic asked in **8 or more** papers · **HOT /41** = **4 to 7** papers · **PIN /41** = **1 to 3** papers but worth ≥ 6 marks.

Mark chips such as **2+6** are the marks that question actually carried. `syllabus, no PYQ yet` marks a syllabus topic never yet examined but written up anyway. `verified/added` marks anything researched and added that no source note in this folder covers.